# Avaloka Assignment

Ankit Kumar Tripathy

## Problem 1: LRU Cache

### Approach

I used a **hash map** together with a **doubly linked list**. The hash map maps each key directly to its node in the list, so I can find any entry in O(1). The list keeps entries ordered by recency: the node next to the head is the most recently used, and the node next to the tail is the least recently used.

A hash map alone cannot track ordering, and a list alone cannot find a key without scanning. Because the map stores a direct reference to each node, and a doubly linked list lets me unlink a node using only its own prev and next pointers, I can move a node to the front or remove the oldest one in constant time. I use sentinel head and tail nodes so I never have to special-case an empty list or the ends of the list.

## Code

```
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity: int):
        if capacity < 0:
            raise ValueError("capacity must be non-negative")
        self.capacity = capacity
        self.map = {}                       # key -> Node
        self.head = Node()                  # sentinel: most-recent side
        self.tail = Node()                  # sentinel: least-recent side
        self.head.next = self.tail
        self.tail.prev = self.head

    # internal helpers, all O(1)
    def _remove(self, node: Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_to_front(self, node: Node) -> None:
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    # public API
    def get(self, key: int) -> int:
        node = self.map.get(key)
        if node is None:
            return -1
        self._remove(node)
        self._add_to_front(node)            # mark as recently used
        return node.value

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return
        node = self.map.get(key)
        if node is not None:                # update existing key
            node.value = value
            self._remove(node)
            self._add_to_front(node)
            return
        if len(self.map) >= self.capacity: # evict LRU item
            lru = self.tail.prev
            self._remove(lru)
            del self.map[lru.key]
        new_node = Node(key, value)
        self.map[key] = new_node
        self._add_to_front(new_node)
```

# Problem 2: Event Scheduler

## Approach for can_attend_all

I sort the events by start time. If every event starts at or after the previous event ends, there are no overlaps. If any event starts strictly before the previous one ends, the person cannot attend both. Since adjacent events (end time equals start time) do not count as overlaps, the check is `start < prev_end`.

## Approach for min_rooms_required

I process events in order of start time and keep a **min-heap of end times**, with one entry per room currently in use. For each new event, if the earliest-ending room has already finished (`heap[0] <= start`), I reuse that room by removing its end time. Then I push the new event's end time. If no room is free, the heap simply grows, which means one more room is needed. The largest size the heap reaches is the minimum number of rooms required, which equals the maximum number of events overlapping at any instant. Using <= gives the required behavior for adjacent events.

## Code

```
import heapq
from typing import List, Tuple

Event = Tuple[int, int]


def can_attend_all(events: List[Event]) -> bool:
    ordered = sorted(events)
    for i in range(1, len(ordered)):
        if ordered[i][0] < ordered[i - 1][1]:
            return False
    return True


def min_rooms_required(events: List[Event]) -> int:
    if not events:
        return 0
    ordered = sorted(events)
    end_times = []                  # min-heap of end times of rooms in use
    max_rooms = 0
    for start, end in ordered:
        if end_times and end_times[0] <= start:
            heapq.heappop(end_times)    # reuse a room that has freed up
        heapq.heappush(end_times, end)
        max_rooms = max(max_rooms, len(end_times))
    return max_rooms
```

## Quick checks

```
can_attend_all([(9, 10), (10, 11), (11, 12)])      # True
can_attend_all([(9, 11), (10, 12)])                # False
min_rooms_required([(9, 10), (10, 11)])            # 1
min_rooms_required([(9, 12), (10, 11), (11, 13)])  # 2
```

### Edge-case tests

```
# LRU cache edge case: zero capacity stores nothing
cache = LRUCache(0)
cache.put(1, 100)
assert cache.get(1) == -1

# Scheduler edge cases
assert can_attend_all([]) is True
assert min_rooms_required([]) == 0
assert min_rooms_required([(9, 10), (10, 11)]) == 1   # adjacent, not overlapping

# Room assignment beyond 26 rooms
many = [(0, 10)] * 30
assert len(set(assign_rooms(many))) == 30
assert room_name(26) == "Room AA"
```

# Final Discussion & Analysis

## 1. Time & Space Complexity

Let *n* be the number of events and *C* the cache capacity.

| Function | Time | Space |
|---|---|---|
| LRUCache.get | O(1) | O(1) extra |
| LRUCache.put | O(1) | O(1) extra |
| LRU cache overall | - | O(C) |
| can_attend_all | O(n log n), dominated by sorting | O(n) for the sorted copy (O(1) extra if sorted in place) |
| min_rooms_required | O(n log n): sort plus n heap operations of O(log n) each | O(n) for the heap and sorted copy |

The O(1) figures for get and put describe time per operation; the cache as a whole uses O(C) space. Hash map lookups are O(1) *on average* (the worst case degrades to O(n) under heavy hash collisions, which Python's hash table makes very unlikely in practice), and linked list pointer updates are O(1) given a direct node reference.

## 2. Trade-offs: Why a hash map plus a doubly linked list?

Each structure covers the other's weakness:

- **Hash map alone** gives O(1) lookup but has no notion of order, so finding the least recently used item would take O(n).
- **Singly linked list or array alone** can track order, but finding a key takes O(n), and removing a node from the middle of a singly linked list requires its predecessor.
- **Together:** the map gives O(1) access to any node, and the doubly linked list lets us unlink that node and reinsert it at the front in O(1), since each node knows both neighbors. Eviction is also O(1) because the least recently used item is always at the tail.

The cost is extra memory (two pointers per node plus hash map overhead) and worse cache locality than an array. In Python, `collections.OrderedDict` provides the same behavior with less code,

but implementing it manually shows the mechanism.

## 3. Future Proofing: Assigning room numbers

I would extend the heap approach to track *which* room is in use, not just how many. I keep two heaps: a **busy heap** of (end_time, room_id) for occupied rooms, and a **free heap** of available room IDs (a min-heap, so the lowest-numbered room is chosen first).

For each event sorted by start time, I pop every busy room whose end_time <= start and move its ID to the free heap. If the free heap has a room, I take it; otherwise I create a new room ID. I assign that room to the event and push (end, room_id) onto the busy heap.

```
import heapq


def room_name(rid: int) -> str:
    """0 -> 'Room A', 25 -> 'Room Z', 26 -> 'Room AA', ... (no upper limit)."""
    n, letters = rid + 1, ""
    while n:
        n, r = divmod(n - 1, 26)
        letters = chr(ord("A") + r) + letters
    return f"Room {letters}"


def assign_rooms(events):
    ordered = sorted(enumerate(events), key=lambda x: x[1])
    busy, free = [], []             # busy: (end_time, room_id); free: room_ids
    next_id = 0
    assignment = {}                 # original index -> room name

    for idx, (start, end) in ordered:
        while busy and busy[0][0] <= start:
            _, rid = heapq.heappop(busy)
            heapq.heappush(free, rid)
        if free:
            rid = heapq.heappop(free)
        else:
            rid = next_id
            next_id += 1
        assignment[idx] = room_name(rid)
        heapq.heappush(busy, (end, rid))

    return [assignment[i] for i in range(len(events))]
```

This keeps the same O(n log n) complexity and uses the minimum number of rooms. To support named rooms with real constraints (capacity, equipment, a fixed room list), the free heap would become a pool of actual room objects, and an event that finds no suitable free room would be rejected or queued. The room_name helper uses spreadsheet-style naming (Room A to Room Z, then Room AA, Room AB, and so on), so there is no cap on the number of rooms.

## 4. Concurrency: Making the LRU Cache thread-safe

Even get modifies shared state because it reorders the list, so both operations are effectively writes and a read-write lock gives no benefit. Without protection, two threads could corrupt the pointers or exceed capacity.

**Simplest fix:** add a mutex and hold it for the entire body of get and put.

```
import threading
```

```
class ThreadSafeLRUCache(LRUCache):
    def __init__(self, capacity):
        super().__init__(capacity)
        self._lock = threading.Lock()

    def get(self, key):
        with self._lock:
            return super().get(key)

    def put(self, key, value):
        with self._lock:
            super().put(key, value)
```

The lock must cover the whole check-then-act sequence (map lookup, list move, eviction), not just individual steps, or a race can occur between them. Since the critical sections are short, this works well at moderate concurrency.

**For higher throughput**, options include:

- **Sharding / lock striping:** split the cache into N independent LRU caches chosen by hash(key) % N, each with its own lock. This reduces contention, but LRU ordering becomes approximate per shard rather than global.
- **Buffered recency updates:** record accesses in a queue and apply the reordering in batches, as high-performance caches such as Caffeine do, so reads do not contend on a lock.

In Python specifically, the GIL does not make compound operations like these atomic, so an explicit lock is still required.